Modo Real vs Modo Protegido

Agenda de hoy

Esta clase es continuación del teórico de modo protegido.

Entorno posible de ejecución de programas x86

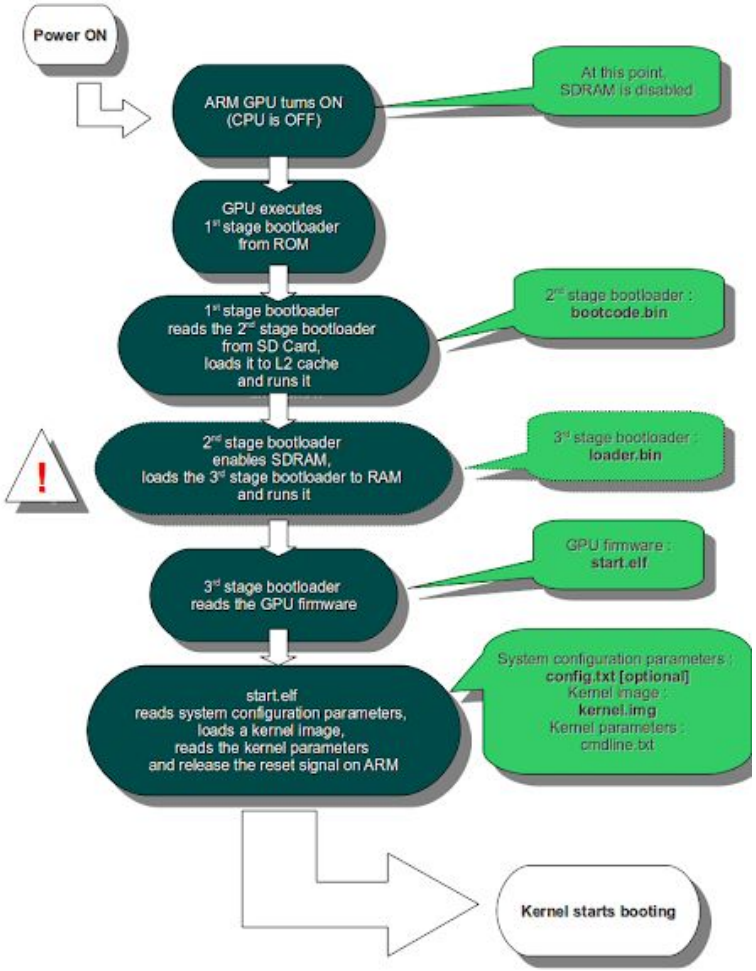
Bios y UEFI

Segmentación

Modo protegido

- GDT descriptor
- GDT

https://gitlab.unc.edu.ar/javierjorge/protected-mode-sdc


Power ON
ARM GPU turns ON
(CPU is OFF)
At this point,
SDRAM is disabled
GPU executes
1st stage bootloader
from ROM
1st stage bootloader
reads the 2nd stage bootloader
from SD Card,
loads it to L2 cache
and runs it
2nd stage bootloader :
bootcode.bin
2nd stage bootloader
enables SDRAM,
loads the 3rd stage bootloader to RAM
and runs it
3rd stage bootloader :
loader.bin
3rd stage bootloader
reads the GPU firmware
GPU firmware :
start.elf
start.elf
reads system configuration parameters,
loads a kernel image,
reads the kernel parameters
and release the reset signal on ARM
System configuration parameters :
config.txt [optional]
Kernel image :
kernel.img
Kernel parameters :
cmdline.txt
Kernel starts booting

Proceso de arranque

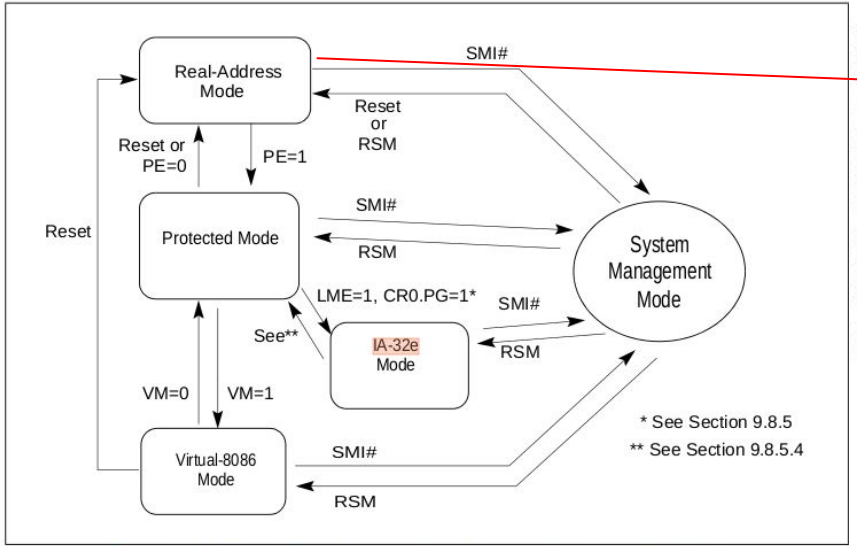
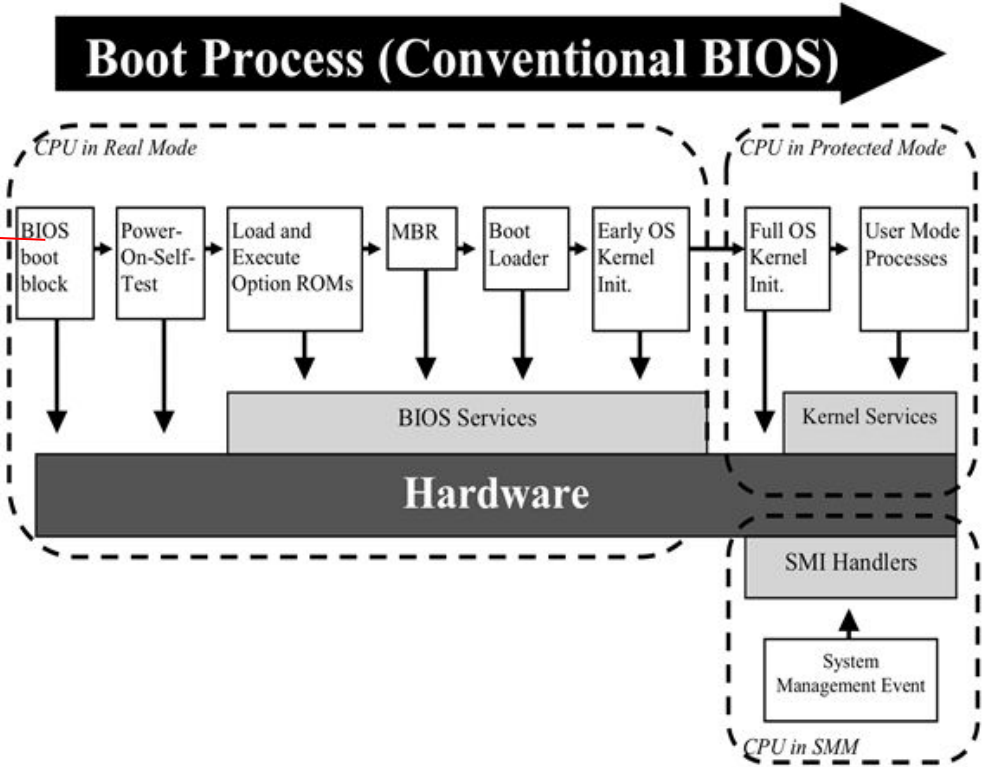


Figure 2-3. Transitions Among the Processor's Operating Modes



Entorno posible de ejecución de programas x86

¿Cómo aprender y practicar?

NASM o GAS

Principales diferencias sintácticas

AT&T and Intel syntax use the **opposite** order for source and destination operands. For example:

Intel: mov eax, 4

AT&T: movl $4, %eax

In AT&T syntax, **immediate operands are preceded by $**; in Intel syntax, immediate operands are not. For example:

Intel: push 4

AT&T: pushl $4

In AT&T syntax, **register operands are preceded by %**; in Intel syntax, they are not.

In AT&T syntax, the size of memory operands is determined from the **last character of the opcode** name. Opcode suffixes of b, w, and l specify byte (8-bit), word (16-bit), and long (32-bit) memory references. Intel syntax accomplishes this by prefixing memory operands (not the opcodes themselves) with byte ptr, word ptr, and dword ptr. Thus:

Intel: mov al, byte ptr foo

AT&T: movb foo, %al

ref: https://developer.ibm.com/technologies/linux/articles/l-gas-nasm/

Como crear una imagen booteable

En la arquitectura x86 lo más simple es crear un sector de arranque MBR y colocarlo en un disco. Se puede crear un sector de arranque con una sola línea de printf

printf '\364%509s\125\252' > main.img

Structure of a classical generic MBR

Address		Description		Size (bytes)
Hex	Dec			
$+000_{hex}$	+0	Bootstrap code area		446
$+1BE_{hex}$	+446	Partition entry №1	*Partition table* (for primary partitions)	16
$+1CE_{hex}$	+462	Partition entry №2		16
$+1DE_{hex}$	+478	Partition entry №3		16
$+1EE_{hex}$	+494	Partition entry №4		16
$+1FE_{hex}$	+510	55_{hex}	*Boot signature*[a]	2
$+1FF_{hex}$	+511	AA_{hex}		
Total size: 446 + 4×16 + 2				**512**

main.img

- **printf** '\364%509s\125\252' > main.img

- \364 in octal == 0xf4 in hex: hlt instruction
- cómo obtener la codificación de una instrucción en particular
- **echo** hlt > a.S
- **as** -o a.o a.S
- **objdump** -S a.o

- %509s produce 509 espacios. Necesarios para completar la imagen hasta el byte 510.

- \125\252 en octal == 0x55 0xAA requisito para que sea interpretada como una mbr

- **hd** main.img

Correr la imagen

instalar y correr qemu con la imagen en cuestión.

sudo apt install qemu-system-x86

qemu-system-x86_64 --drive file=**main.img**,format=**raw**,index=0,media=**disk**

Ejecutar programas en el hardware

Ejecutaremos programas directamente sobre el HW

- usar pc con un usb
- Virtualizar

Correr en HW

Grabar un pendrive con la imagen a probar

sudo dd if=main.img of=/dev/sdX

colocar el pen en la pc

encenderla e indicar que inicie desde la misma.

BIOS/UEFI

Viejo conocido

BIOS

Solo se puede acceder en modo real

Es vieja, pero es uno de los firmware mejor conocidos

UEFI es el nuevo estándar

Las funciones de la BIOS solo se acceden mediante interrupciones y los argumentos se pasan por registros

https://en.wikipedia.org/wiki/BIOS_interrupt_call#Interrupt_table

Desafío: UEFI y coreboot (40 min)

¿Qué es UEFI? ¿como puedo usarlo? Mencionar además una función a la que podría llamar usando esa dinámica.

¿Menciona casos de bugs de UEFI que puedan ser explotados?

¿Qué es Converged Security and Management Engine (CSME), the Intel Management Engine BIOS Extension (Intel MEBx).?

¿Qué es coreboot ? ¿Qué productos lo incorporan ?¿Cuales son las ventajas de su utilización?

Pequeño hello world

main.S

```
.code16
    mov $msg, %si
    mov $0x0e, %ah
loop:
    lodsb
    or %al, %al
    jz halt
    int $0x10
    jmp loop
halt:
    hlt
msg:
    .asciz "hello world"
```

link.ld

```
SECTIONS
{
    /* The BIOS loads t
     * We must tell tha
     * calculate the ac
     */
    . = 0x7c00;
    .text :
    {
        __start = .;
        *(.text)
        /* Place the ma
        . = 0x1FE;
        SHORT(0xAA55)
    }
}
```

compilar y linkear

```
as -g -o main.o main.S
ld --oformat binary -o main.img -T link.ld main.o
qemu-system-x86_64 -hda main.img
```

https://stackoverflow.com/questions/59881880/what-memory-is-impacted-using-the-location-counter-in-linker-script
https://www.math.utah.edu/docs/info/ld_3.html#SEC3
https://www.glamenv-septzen.net/en/view/6

Desafío: Linker

Crear un documento donde respondan a las siguientes preguntas

¿Que es un linker? ¿que hace ?

¿Que es la dirección que aparece en el script del linker?¿Porqué es necesaria ?

Compare la salida de objdump con hd, verifique donde fue colocado el programa dentro de la imagen.

Grabar la imagen en un pendrive y probarla en una pc y subir una foto

¿Para que se utiliza la opción --oformat binary en el linker?

Depuración de ejecutables con llamadas a bios int

```
# gdb
GNU gdb (GDB) 7.6.1-ubuntu
[...]
(gdb) target remote localhost:1234
Remote debugging using localhost:12
0x0000fff0 in ?? ()
(gdb) set architecture i8086
[...]
(gdb) br *0x7c00
```

[gdb dashboard]

una vez lanzado qemu se puede liberar el mouse con **ctrl+alt**

qemu-system-i386 -fda ../01HelloWorld/main.img

-boot a -s -S -monitor stdio

depurar con gdb:

- colocar un breakpoint en la dirección de arranque,
- luego colocar otro a continuación de la llamada a la interrupción
- Utilizar “c” continue antes de las interrupciones y “si” para ejecutar una sola instrucción.

https://stackoverflow.com/questions/24491516/how-to-step-over-interrupt-calls-when-debugging-a-bootloader-bios-with-gdb-and-q

Modo protegido

recuerden estudiar la clase teórica

Modos de funcionamiento x86

- Real-address, "real mode"
- Protected
- System management
- IA-32e. Has two sub modes:
  Compatibility
  64-bit

Real mode, protected mode, virtual 8086 mode, and system management mode. These are sometimes referred to as legacy modes.

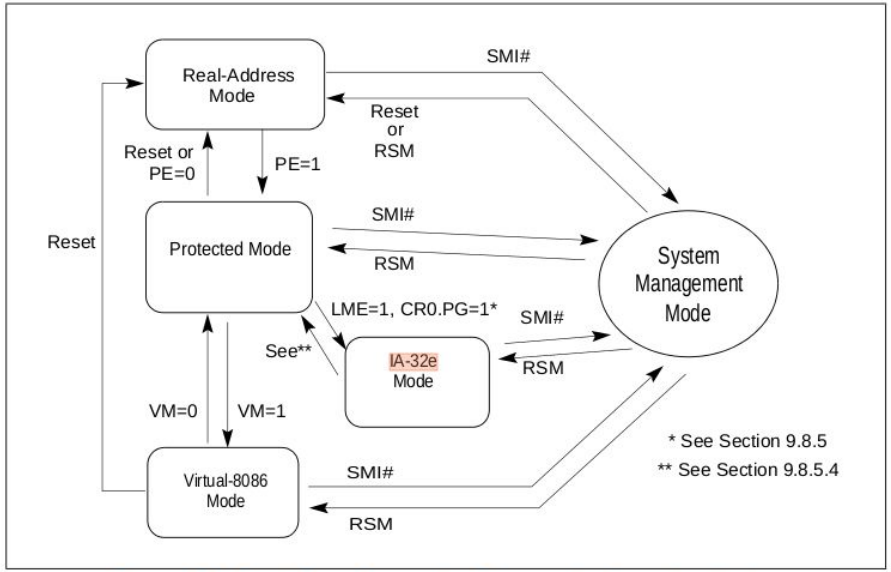


Figure 2-3. Transitions Among the Processor's Operating Modes

Modelos de memoria

Segmentación

Paginación

nota: La arquitectura x86-64 no usa segmentación

Más información sobre registros, **CR0**

Control registers in x64 series [edit]

CR0 [edit]

The CR0 register is 32 bits long on the 386 and higher processors. On x86-64 processors in long mode, it (and the other control registers) is 64 bits long. CR0 has various control flags that modify the basic operation of the processor.

Bit	Name	Full Name	Description
0	PE	Protected Mode Enable	If 1, system is in protected mode, else system is in real mode
1	MP	Monitor co-processor	Controls interaction of WAIT/FWAIT instructions with TS flag in CR0
2	EM	Emulation	If set, no x87 floating-point unit present, if clear, x87 FPU present
3	TS	Task switched	Allows saving x87 task context upon a task switch only after x87 instruction used
4	ET	Extension type	On the 386, it allowed to specify whether the external math coprocessor was an 80287 or 80387
5	NE	Numeric error	Enable internal x87 floating point error reporting when set, else enables PC style x87 error detection
16	WP	Write protect	When set, the CPU can't write to read-only pages when privilege level is 0
18	AM	Alignment mask	Alignment check enabled if AM set, AC flag (in EFLAGS register) set, and privilege level is 3
29	NW	Not-write through	Globally enables/disable write-through caching
30	CD	Cache disable	Globally enables/disable the memory cache
31	PG	Paging	If 1, enable paging and use the § CR3 register, else disable paging.

Segmentación en modo real

Ver Ejemplo
https://github.com/cirosantilli/x86-bare-metal-examples/tree/18772b1403133b2328d5ad44791445f9859de320#real-mode-segmentation

FS y GS no tienen usos asignados por hardware. El Linux kernel usa GS para poner datos por cada CPU.

Más segmentación

En general los segmentos se alteran de manera indirecta mediante otro registro o con instrucciones propias.

CS se altera con ljmp

SS afecta instrucciones que usen el SP como PUSH and POP (16 * SS + SP)

Modo protegido

Bios ya no está disponible

Utilizar VGA para salidas

Es necesario crear una GDT para arrancar

Las instrucciones dejan de ser de 16bits para ser de 32bits (.code32)

Permite el uso de anillos o rings de seguridad.

Proceso:

Deshabilitar interrupciones

Cargar la GDT

Fijar el bit más bajo del CR0 en 1

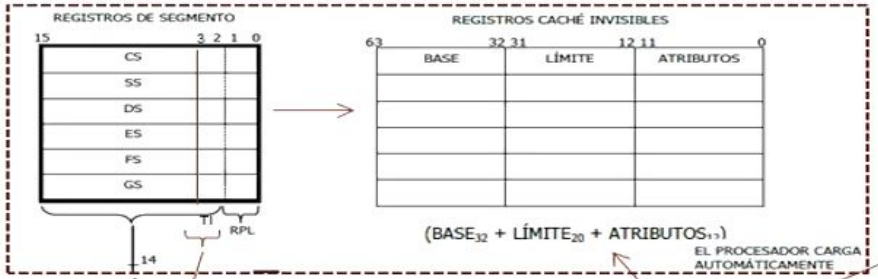
saltar a la sección de código de 32bits

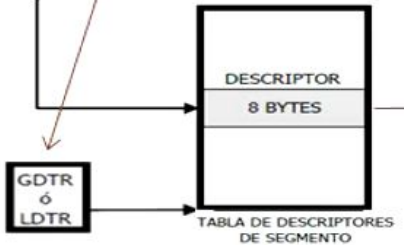
Configurar el resto de los segmentos

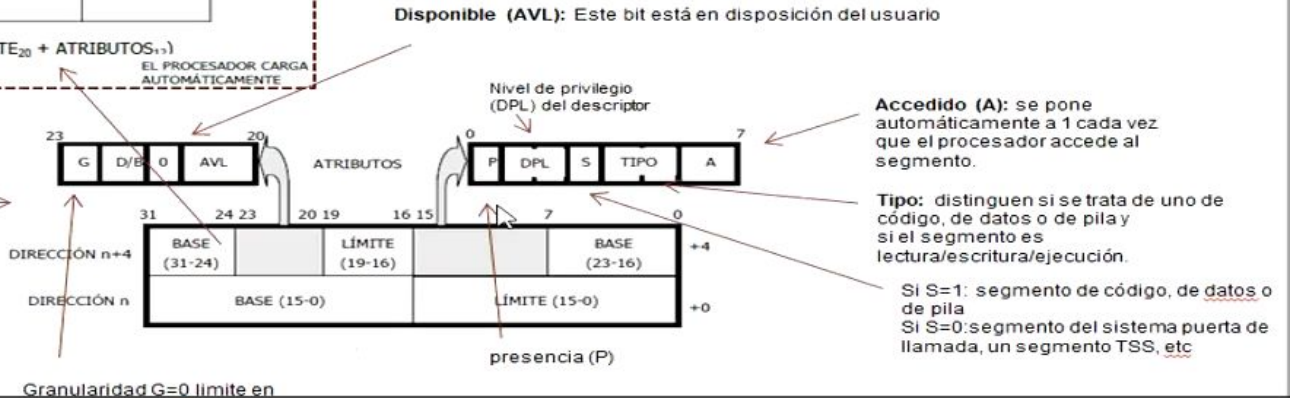
- Modo Protegido:
  - Un selector señala a un descriptor de segmento que contiene: Base de 32 bits, Límite de 20 bits y Atributos de 12 bits.

Los componentes del descripto los usa Unidad de Segmentación:

1. Si la dirección está dentro del tamaño del segmento.
2. Si el segmento está presente en la memoria.
3. Si el nivel de privilegio del segmento hace posible su acceso.
4. Si se puede leer, escribir o ejecutar.







GDT descriptor y GDT

The GDT Descriptor

Bits	Function	Description
0-15	Limit	Size of GDT in bytes
16-47	Address	GDT's memory address

1st Double word:

Bits	Function	Description
0-15	Limit 0:15	First 16 bits in the segment limiter
16-31	Base 0:15	First 16 bits in the base address

2nd Double word:

Bits	Function	Description
0-7	Base 16:23	Bits 16-23 in the base address
8-12	Type	Segment type and attributes
13-14	Privilege Level	0 = Highest privilege (OS), 3 = Lowest privilege (User applications)
15	Present flag	Set to 1 if segment is present
16-19	Limit 16:19	Bits 16-19 in the segment limiter
20-22	Attributes	Different attributes, depending on the segment type
23	Granularity	Used together with the limiter, to determine the size of the segment
24-31	Base 24:31	The last 24-31 bits in the base address

http://www.osdever.net/tutorials/view/the-world-of-protected-mode

Proceso simplificado

/* Tell the processor where our Global Descriptor Table is in memory. */

lgdt gdt_descriptor

/ Set PE (Protection Enable) bit in CR0 (Control Register 0)*/*

mov %cr0, %eax

orl $0x1, %eax

mov %eax, %cr0

ljmp $CODE_SEG, $protected_mode

.code32

protected_mode:

/* Setup the other segments * Those movs are mandatory because they update the descriptor cache: * http://wiki.osdev.org/Descriptor_Cache

*/

mov $DATA_SEG, %ax

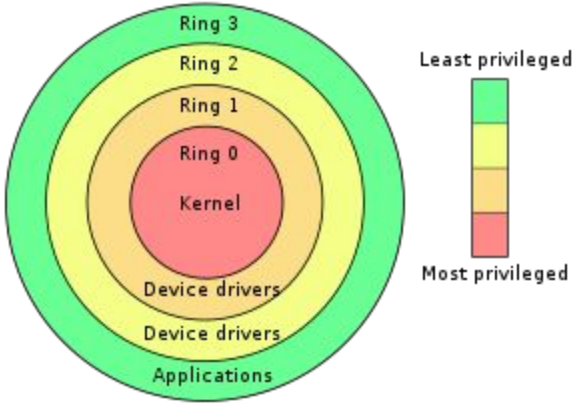
...

Ver ejemplo impresion en memoria de video

https://github.com/cirosantilli/x86-bare-metal-examples/blob/master/common.h#L135

```
#include "common.h"
BEGIN
    CLEAR
    PROTECTED_MODE
    VGA_PRINT_STRING $message
    jmp .
message:
    .asciz "hello world"
```

Seguridad Anillos



Ejemplo para verificar junto con la sección de módulos de kernel

https://github.com/cirosantilli/linux-kernel-module-cheat/tree/ed5fa984c6226f81cb1a07f980d319ee9ee88e00#ring0

Desafío final: Modo protegido

Crear un código assembler que pueda pasar a modo protegido (sin macros).

¿Cómo sería un programa que tenga dos descriptores de memoria diferentes, uno para cada segmento (código y datos) en espacios de memoria diferenciados?

Cambiar los bits de acceso del segmento de datos para que sea de solo lectura, intentar escribir, ¿Que sucede? ¿Que debería suceder a continuación? (revisar el teórico) Verificarlo con gdb.

En modo protegido, ¿Con qué valor se cargan los registros de segmento ? ¿Porque?

Donde seguir

Otro punto de vista del mismo tema
http://sistemasdecomputacionunc.blogspot.com/2014/04/paso-modo-protegido-x86.html

https://wiki.osdev.org/GDT_Tutorial

Explicacion sobre el código de actualización de los segmentos y la tabla GDT
https://stackoverflow.com/questions/23978486/far-jump-in-gdt-in-bootloader

Referencia para la creación de las filminas
http://www.osdever.net/tutorials/view/the-world-of-protected-mode

Donde encotrar código del kernel de linux sobre la GDT
https://stackoverflow.com/questions/25762625/file-in-which-the-data-structure-for-global-descriptor-and-local-descriptor-tabl

Compartir y usar esta presentación

ref: https://github.com/cirosantilli/linux-kernel-module-cheat

ref: https://github.com/cirosantilli/x86-bare-metal-examples

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.